

Production-Style OTP Notification + Verification Flow

Comprehensive Technical Implementation & Architecture Verification Report

Architecture: Spring Boot Microservices • Apache Kafka • Redis • SAGA Pattern • React Dashboard

Executive Summary: Implemented an end-to-end, production-ready Two-Factor OTP security verification workflow triggered by the Fraud Detection engine. Follows strict distributed system guardrails: Fraud Detection evaluates risk, Transaction Service manages OTP lifecycles in Redis, Notification Service delivers multichannel alerts (Email + SMS), and the React Frontend provides a live verification modal with refresh resilience.

1. Files Changed & Created

■ Backend Services (backend/)

- **backend/transaction-service/.../TransactionService.java:** Added verifyOTP validation against Redis with verification attempt rate-limiting (max 5 attempts). Maintained PENDING_VERIFICATION on wrong/expired OTPs without premature cancellation. Implemented resendOTP with rate limiting (max 3 resends per 10 min), old OTP invalidation, SecureRandom generation, Redis TTL reset, and Kafka notification dispatch. Connected verified OTPs directly to the Saga engine (completeTransaction → transaction.completed topic).
- **backend/transaction-service/.../TransactionEventConsumer.java:** Integrated cryptographically secure 6-digit OTP generation for incoming verification.required events. Stored OTP in Redis under verification:otp{transactionId} with 5-min TTL. Injected AccountServiceClient to fetch registered email/phone. Published enriched transaction.otp.generated Kafka event.
- **backend/transaction-service/.../TransactionController.java:** Added POST /api/v1/transactions/{transactionId}/resend-otp endpoint and exposed verify OTP endpoint.
- **backend/transaction-service/.../AccountServiceClient.java:** Added getAccount(accountNumber) Feign client method to query Account Service for customer contact details.
- **backend/transaction-service/.../AccountResponse.java (Created):** DTO containing registered customer profile attributes (email, phone, account type, balance).
- **backend/transaction-service/.../GlobalExceptionHandler.java (Created):** Standardized JSON error responses for ResponseStatusException (400, 429, 500) ensuring clear client-facing error messages.
- **backend/transaction-service/.../TransactionServiceOtpTests.java (Created):** Comprehensive unit test suite for OTP verification, invalid codes, expiration, attempt rate limiting, and resend quotas.
- **backend/transaction-service/.../TransactionEventConsumerTests.java (Created):** Unit tests verifying Kafka consumption of verification.required, Redis persistence, and notification dispatching.
- **backend/notification-service/.../EmailNotificationService.java (Created):** Interface defining email notification abstractions.
- **backend/notification-service/.../EmailNotificationServiceImpl.java (Created):** Production-ready SMTP mail service using JavaMailSender with automatic simulation fallback when host is unconfigured.
- **backend/notification-service/.../SmsNotificationService.java (Created):** Interface defining SMS provider delivery abstractions.
- **backend/notification-service/.../SmsNotificationServiceImpl.java (Created):** SMS delivery service supporting Twilio/AWS SNS/Mock providers with phone masking and fallback simulation.
- **backend/notification-service/.../NotificationService.java:** Updated transaction.otp.generated listener to dispatch email & SMS concurrently without leaking raw OTPs in logs.
- **backend/notification-service/.../application.yaml:** Configured SMTP and SMS provider environment variable bindings with sensible defaults.
- **backend/notification-service/.../NotificationServiceTests.java (Created):** Unit tests validating email, SMS, and fallback notification handling.

■ Frontend React Application (frontend/)

- **frontend/src/api/axios.js:** Added resendOtp(transactionId) to the centralized transactionApi object.

- **frontend/src/pages/Transfer.jsx:** Implemented full production OTP Verification modal with masked contact display (s****@gmail.com, +91 *****1234), 6-digit numeric input, paste support, 5-minute countdown (05:00), [Resend OTP] button with cooldown, in-app toast stack, PROCESSING status polling, and browser refresh state recovery via sessionStorage.
- **frontend/src/styles/forms.css:** Added styling rules for toast notifications, masked contact badges, countdown timer, and resend button.

2. Backend Architecture & SAGA Event Flow

- 1. Transfer Initiation:** Client calls POST /api/v1/transactions/transfer. Transaction Service deducts sender balance via Feign, sets status to PROCESSING, and publishes transaction.initiated to Kafka.
- 2. Risk Evaluation:** Fraud Detection Service evaluates velocity (>5/min), amount (>5x average), and balance (>90%). If suspicious, it publishes verification.required.
- 3. OTP Generation & Storage:** Transaction Service consumes verification.required, generates a secure 6-digit OTP using SecureRandom, saves it to Redis (verification:otp{txId}, TTL 5m), sets status to PENDING_VERIFICATION, fetches contact data, and publishes transaction.otp.generated.
- 4. Notification Delivery:** Notification Service consumes the event and dispatches the OTP via EmailNotificationService and SmsNotificationService with masked audit logs.
- 5. Verification & Saga Completion:** Client sends OTP to POST /api/v1/transactions/{id}/verify?otp=.... Transaction Service verifies against Redis. If valid, it deletes the OTP, marks transaction COMPLETED, and publishes transaction.completed. Account Service then credits the receiver.

3. Frontend Changes & User Experience

- OTP Verification Modal:** Clean, accessible modal showing transaction amount, masked contact details, 6-digit numeric input with auto-focus and clipboard paste, and disabled verify button until 6 digits are typed.
- In-App Toast Alerts:** Sequential real-time feedback: *'Transaction verification required. OTP sent to your registered contact.'* → *'OTP sent successfully.'* → *'Transaction verified successfully.'* → *'Transaction completed successfully.'*
- Background Polling:** When a transaction is submitted, frontend polls GET /api/v1/transactions/{id} every 2.5 seconds to seamlessly detect when the state transitions from PROCESSING to PENDING_VERIFICATION or COMPLETED.
- Browser Refresh Resilience (Test 7):** Active pending transaction IDs are saved in sessionStorage (apex_pending_otp_tx_id). On page reload, the frontend automatically retrieves the transaction state and reopens the OTP modal without losing progress.

4. Kafka Events & Topics Used

Topic Name	Producer	Consumer(s)	Payload Structure
transaction.initiated	transaction-service	fraud-detection-service	{ transactionId, senderAccountNumber, receiverAccountNumber, amount, description }
verification.required	fraud-detection-service	transaction-service	{ transactionId, accountNumber, amount, reason }
fraud.check.clean	fraud-detection-service	transaction-service	{ transactionId, isFailed: false, reason: null }
transaction.otp.generated	transaction-service	notification-service	{ transactionId, accountNumber, email, phoneNumber, notificationType: 'OTP', message, expiresInMinutes: 5, otp, amount, reason }
transaction.completed	transaction-service	account-service, notification-service	{ transactionId, senderAccountNumber, receiverAccountNumber, amount, description }
transaction.refunded	transaction-service	notification-service	{ transactionId, senderAccountNumber, amount, reason }
fraud.detected	transaction-service	account-service, notification-service	{ transactionId, accountNumber, reason }

5. Redis Keys & TTL

Redis Key	Purpose	TTL	Lifecycle
verification:otp{transactionId}	Stores generated 6-digit OTP for verification	5 minutes (300s)	Deleted on successful verification or invalidated on resend
verification:attempts:{transactionId}	Tracks failed OTP verification attempts (max 5)	5 minutes (300s)	Reset on valid OTP entry or expired after 5m
verification:resend_count:{transactionId}	Tracks OTP resend requests (max 3 allowed)	10 minutes (600s)	Rate limits spamming; resets after 10 minutes
fraud:velocity{accountNumber}	Velocity fraud counter (max 5 tx per minute)	60 seconds	Auto-increments on each transaction initiation
fraud:avg_amount{accountNumber}	Tracks historical moving average transaction amount	Persistent	Updated on each clean transfer evaluation

6. Email Configuration & 7. SMS Provider Configuration

Email Notification Specification Subject: Digital Bank - Transaction Verification OTP Body Template: Your Digital Bank transaction requires verification. Amount: \$60.00 OTP: 482931 This OTP expires in 5 minutes. If you did not initiate this transaction, contact support immediately.	SMS Notification Specification SMS Message Format: Digital Bank OTP: 482931 is your verification code for amount \$60.00. Valid for 5 minutes. If you did not initiate this transaction, contact support immediately. Supported Providers: • mock (Default local development; logs masked dispatch without external network calls) • twilio / aws-sns (Configurable via environment variables)
---	---

8. Environment Variables Required

```
# =====
# Notification Service - Email Configuration
# =====
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_USERNAME=your-email@gmail.com
MAIL_PASSWORD=your-app-password
MAIL_FROM=no-reply@digitalbank.com
MAIL_SMTP_AUTH=true
MAIL_STARTTLS=true

# =====
# Notification Service - SMS Configuration
# =====
SMS_PROVIDER=mock
SMS_API_KEY=
SMS_API_SECRET=
SMS_FROM=+10000000000

# =====
# Database & Infrastructure
# =====
DB_USERNAME=root
DB_PASSWORD=root
```

9. Exact API Endpoints

Method	Endpoint (via Gateway :8080)	Description
POST	/api/v1/transactions/transfer	Initiate money transfer (Saga Step 1)
GET	/api/v1/transactions/{transactionId}	Retrieve real-time transaction status & audit metadata
GET	/api/v1/transactions/account/{accountNumber}	Get full transaction history for an account
POST	/api/v1/transactions/{transactionId}/verify?otp={otp}	Validate 6-digit OTP against Redis and complete Saga
POST	/api/v1/transactions/{transactionId}/resend-otp	Generate new secure OTP with rate limiting and reset 5m TTL
GET	/api/v1/accounts/{accountNumber}	Get account profile (email, phone, status, limit)
GET	/api/v1/accounts/{accountNumber}/balance	Get real-time balance for account

10. Verification & Test Execution Results

```
[INFO] Running com.banking.transactionservice.service.TransactionEventConsumerTests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.banking.transactionservice.service.TransactionServiceOtpTests
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.banking.notification.service.NotificationServiceTests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS (11 of 11 tests passing)
[INFO] -----

Frontend Vite Production Build:
vite v6.4.3 building for production...
✓ 105 modules transformed.
dist/index.html          0.95 kB
dist/assets/index-CAq_3nJ6.css  21.72 kB
dist/assets/index-Bpa6pcdU.js  278.26 kB
✓ built in 3.06s (0 errors, 0 warnings)
```

11. End-to-End Testing Guide

- **Test 1: Normal Transfer (\$2.00):** Amount below threshold → Transaction completes immediately (COMPLETED). No OTP popup.
- **Test 2: Suspicious Transfer (\$60.00):** Amount triggers multiplier rule → Status transitions to PENDING_VERIFICATION. OTP modal appears with masked contact details (s****@gmail.com, +91 *****1234). OTP stored in Redis and dispatched via Kafka.
- **Test 3: Enter Valid OTP:** User enters 6 digits and clicks Verify → Backend deletes Redis OTP key, completes Saga (COMPLETED), credits receiver, and refreshes UI balance.
- **Test 4: Enter Invalid OTP:** User enters wrong OTP (e.g. 000000) → Backend returns 400 Bad Request: 'Invalid OTP'. Transaction safely remains in PENDING_VERIFICATION.
- **Test 5: OTP Expiry:** After 5 minutes (or Redis key expiry) → Verification returns 'OTP expired. Please request a new OTP.'. Transaction remains in PENDING_VERIFICATION.
- **Test 6: Resend OTP:** User clicks [Resend OTP] → Previous OTP invalidated, new 6-digit OTP generated, TTL reset to 5 min, notification re-sent, countdown resets to 05:00. New OTP succeeds.
- **Test 7: Browser Refresh Resilience:** User refreshes browser during PENDING_VERIFICATION → Frontend restores transaction ID from sessionStorage and reopens verification modal seamlessly.

12. External Email/SMS Provider Limitations & Fallbacks

- **Unconfigured / Localhost SMTP:** If MAIL_HOST is empty or set to localhost without a running mail server, the system automatically falls back to simulated email delivery and logs a masked dispatch warning without failing the microservice.
- **Unconfigured SMS Gateway:** If SMS_PROVIDER is set to mock or credentials are omitted, SMS delivery is simulated cleanly with masked phone logging.
- **Delivery Failures:** If one channel fails (e.g., SMS network error), the user can still receive email OTP. If all channels fail, the transaction remains safely in PENDING_VERIFICATION so the user can retry via [Resend OTP] without financial loss.
- **Security Guarantee:** Raw OTPs are NEVER exposed in production logs or frontend JSON response payloads.